PYTHON KURSU • HAFTA 10 • GÜN 2

Pandas 1: Tablolarla Tanışma

Ders 20 / 32 • 3 saat • VS Code + Python 3.12

Bu dersin sonunda…

- Pandas’ın ne işe yaradığını ve NumPy’den farkını açıklayabileceksiniz.
- Series ve DataFrame oluşturup index ve columns kavramlarını kullanabileceksiniz.
- head, info, describe ile bir tabloyu hızla tanıyabileceksiniz.
- Sütun seçip loc ve iloc ile satır ve hücrelere ulaşabileceksiniz.
- Yeni sütun ekleyip sort_values ve value_counts ile basit analiz yapabileceksiniz.

AÇILIŞ • 10 DK

Bugünün zaman planı

Saat	Süre	Bölüm
0:00–0:10	10 dk	Açılış: hedefler, Ders 19'in tekrarı
0:10–1:25	75 dk	Blok 1: Pandas nedir • kurulum • Series • Series ile işlem • DataFrame oluşturmanın üç yolu • index ve columns • veri tipleri
1:25–1:40	15 dk	Mola
1:40–2:55	75 dk	Blok 2: head, info, describe • sütun seçme • loc ve iloc • yeni sütun • set_index • sort_values, value_counts • kitaplık analizi • alıştırmalar
2:55–3:00	5 dk	Ödev ve gelecek ders

AÇILIŞ • DERS 19'IN TEKRARI • 6 DK

Dört soruda geçen ders

SORU 1 • Neden hata?

```
a[(a > 125) and (a < 140)]
```

SORU 2 • Ne hesaplar?

```
adet @ fiyat
# adet (3, 3), fiyat (3,)
```

SORU 3 • Ne verir?

```
np.where(satis ⩾ 125,
         'Tuttu', 'Kaldı')
```

SORU 4 • Ne değişir?

```
ilk = satis[:2]
ilk[0] = 0
# satis?
```

AÇILIŞ • ÖDEV KONTROLÜ • 4 DK

Ödev 1: kantin satışları

```
import numpy as np

urunler = np.array(["Simit", "Tost", "Ayran", "Su", "Kek"])
fiyat = np.array([15.0, 45.0, 20.0, 10.0, 25.0])
# satırlar: günler (Pzt-Cum) · sütunlar: ürünler
adet = np.array([[40, 22, 30, 50, 12], [35, 25, 28, 45, 0],
                 [42, 30, 35, 60, 15], [38, 18, 25, 40, 10],
                 [50, 35, 40, 70, 20]])
print("Günlük ciro:", adet @ fiyat)
print("En çok satan:", urunler[adet.sum(axis=0).argmax()])
```

```
ÇIKTI
Günlük ciro: [2990. 2660. 3655. 2530. 4325.]
En çok satan: Su
```

BLOK 1 • 0:10–1:25 • 75 DK

Series ve DataFrame

12 dk
Pandas nedir?

20 dk
Series

28 dk
DataFrame

15 dk
Veri tipleri ve hatalar

BLOK 1 • PANDAS NEDIR? • 12 DK

Pandas: Python’da Excel tablosu

Benzetme: etiketli NumPy

NumPy dizisi numaralı rafları olan bir depo: “3. raf, 2. kutu”. Pandas aynı depoya etiket yapıştırır: “Çankaya rafı, Mart kutusu”. Veriler yine NumPy’de durur; Pandas adları ve kolaylıkları ekler.

	NumPy	Pandas
Veri	sayı dizileri	tablolar (karışık tipler)
Adlar	yok (konum)	satır ve sütun adları
Sütun tipi	tek tip	her sütun kendi tipinde
Dosya	sınırlı	CSV, Excel, SQL, JSON…
Eksik veri	np.nan	NaN araçları (isna, fillna)

BLOK 1 • PANDAS NEDIR? • 12 DK

Kurulum

```
# Terminal (sanal ortam etkinken)
python -m pip install pandas
```

```
import pandas as pd

print(pd.__version__)
```

```
ÇIKTI
3.1.0
```

Pandas, NumPy’yi de birlikte kurar. Sürüm numaranız farklı olabilir; 2.x ve 3.x bu dersteki örneklerin hepsini çalıştırır.

BLOK 1 • SERIES • 20 DK

Series: adları olan tek boyutlu dizi

```
import pandas as pd

notlar = pd.Series([72, 45, 88, 91], index=["Ece", "Mert", "Zeynep", "Can"])
print(notlar)
print(notlar["Zeynep"], notlar.iloc[0])
print(notlar.index.tolist(), notlar.values)
```

```
ÇIKTI
Ece       72
Mert      45
Zeynep    88
Can       91
dtype: int64
88 72
['Ece', 'Mert', 'Zeynep', 'Can'] [72 45 88 91]
```

BLOK 1 • SERIES • 20 DK

Series ile işlem: konuma göre değil, ada göre eşleşir

```python
import pandas as pd

vize = pd.Series([60, 75, 40], index=["Ece", "Mert", "Zeynep"])
final = pd.Series([85, 70, 55], index=["Mert", "Ece", "Can"])
print((vize * 0.4 + final * 0.6).round(1))
print(vize[vize ⩾ 50])
print(vize.mean(), vize.max(), vize.idxmax())
```

```
ÇIKTI
Can        NaN
Ece       66.0
Mert      81.0
Zeynep     NaN
dtype: float64
Ece     60
Mert    75
dtype: int64
58.333333333333336 75 Mert
```

Önemli fark

NumPy konumla eşleştirirdi. Pandas index adlarıyla eşleştirir: Mert'in vizesi Mert'in finaliyle toplanır. Bir tarafta olmayan ad (Can, Zeynep) NaN olur.

BLOK 1 • SERIES • 20 DK

Sözlükten Series

```
import pandas as pd

nufus = pd.Series({"İstanbul": 15.7, "Ankara": 5.8, "İzmir": 4.5})
print(nufus)
print("Ankara" in nufus, nufus.sum().round(1))
nufus["Bursa"] = 3.2            # yeni eleman
print(nufus.sort_values(ascending=False))
```

```
ÇIKTI
İstanbul    15.7
Ankara       5.8
İzmir        4.5
dtype: float64
True 26.0
İstanbul    15.7
Ankara       5.8
İzmir        4.5
Bursa        3.2
dtype: float64
```

Sözlüğün anahtarları index, değerleri değerler olur. Rakamlar milyon kişi, yuvarlanmış yaklaşık değerlerdir.

BLOK 1 • DATAFRAME • 28 DK

DataFrame: sütunları adlandırılmış tablo

```python
import pandas as pd

kitaplar = pd.DataFrame({
    "ad": ["Tutunamayanlar", "İnce Memed", "Semaver"],
    "yazar": ["Oğuz Atay", "Yaşar Kemal", "Sait Faik"],
    "yil": [1972, 1955, 1936],
    "puan": [5, 4, 3],
})
print(kitaplar)
```

```
ÇIKTI
                ad         yazar   yil  puan
0  Tutunamayanlar     Oğuz Atay  1972     5
1      İnce Memed   Yaşar Kemal  1955     4
2         Semaver     Sait Faik  1936     3
```

Kalıp

{“sütun adı”: [değerler listesi], …}. Her anahtar bir sütun, her liste o sütunun değerleri. Bütün listeler aynı uzunlukta olmalı.

BLOK 1 • DATAFRAME • 28 DK

DataFrame’in parçaları: index, columns, değerler

```
print(kitaplar.shape)
print(kitaplar.columns.tolist())
print(kitaplar.index)
print(type(kitaplar["yil"]))
```

```
ÇIKTI
(8, 6)
['ad', 'yazar', 'yil', 'tur',
  'puan', 'okundu']
RangeIndex(start=0, stop=8,
  step=1)
<class 'pandas.Series'>
```

Parça	Ne?	Örnek
`index`	satır adları	`0 … 7 (ya da kitap adları)`
`columns`	sütun adları	`ad, yazar, yil, tur, puan, okundu`
`shape`	(satır, sütun)	`(8, 6)`
`her sütun`	bir Series	`kitaplar["yil"]`

BLOK 1 • DATAFRAME • 28 DK

DataFrame oluşturmanın iki yolu daha

```
import numpy as np
import pandas as pd

# 1) Sözlük listesinden (her sözlük bir satır)
kayitlar = [{"ad": "Kalem", "fiyat": 12.5, "stok": 40},
            {"ad": "Defter", "fiyat": 45.0, "stok": 10}]
urunler = pd.DataFrame(kayitlar)
print(urunler)

# 2) NumPy matrisinden (Ders 19)
satis = np.array([[120, 135, 128, 150],
                  [ 98, 110, 105, 120],
                  [140, 138, 152, 160]])
df = pd.DataFrame(satis, index=["Kadıköy", "Bornova", "Çankaya"],
                  columns=["Oca", "Şub", "Mar", "Nis"])
print(df)
```

ÇIKTI

```
       ad  fiyat  stok
0   Kalem   12.5    40
1  Defter   45.0    10
         Oca  Şub  Mar  Nis
Kadıköy  120  135  128  150
Bornova   98  110  105  120
Çankaya  140  138  152  160
```

Sütun veri tipleri

```
print(kitaplar.dtypes.astype(str).to_dict())
kitaplar["yil"] = kitaplar["yil"].astype(float)
print(kitaplar["yil"].dtype)
tarih = pd.to_datetime(pd.Series(["2026-10-20", "2026-10-27"]))
print(tarih.dt.day_name().tolist())
```

```
ÇIKTI
{'ad': 'str', 'yazar': 'str', 'yil': 'int64', 'tur': 'str', 'puan': 'int64', 'okundu': 'bool'}
float64
['Tuesday', 'Tuesday']
```

BLOK 1 • VERI TIPLERI VE HATALAR • 15 DK

Hangi sütun hangi tipte olmalı?

dtype	Ne için?	Örnek sütun
`int64`	tamsayı	yil, puan
`float64`	ondalık (NaN da)	fiyat, ortalama
`str (Pandas 2: object)`	metin	ad, yazar, plaka
`bool`	True / False	okundu
`datetime64`	tarih-saat	satış tarihi
`category`	az sayıda tekrar eden değer	tur, şehir

Kural

Üzerinde hesap yapılacaksa sayı; kimlik, kod, telefon gibi hesap yapılmayacaksa metin. Tarihler her zaman datetime'a çevrilir.

BLOK 1 • SIK YAPILAN HATALAR

Series ve DataFrame oluştururken sık hatalar

Hatalı kod	Hata mesajı	Çözüm
`pd.DataFrame({"ad": [3 eleman], "puan": [2 eleman]})`	`ValueError: All arrays must be of the same length`	listeler aynı uzunlukta
`notlar["Ali"]`	`KeyError: 'Ali'`	ad index'te mi? in ile
`notlar.iloc[5]`	`IndexError: single positional indexer is out-of-bounds`	konum 0 … len−1
`pd.series([1, 2])`	`AttributeError: module 'pandas' has no attribute 'series'. Did you mean: 'Series'?`	büyük harf: pd.Series

1:25–1:40 • 15 DK

Mola

Moladan sonra: 8 kitaplık tabloyu tanıyalım, en yüksek puanlıları sıralayalım.

BLOK 2 • 1:40–2:55 • 75 DK

Tabloyu tanımak ve kullanmak

15 dk
İnceleme

20 dk
Seçmek

20 dk
Değiştirmek ve sıralamak

20 dk
Uygulama ve alıştırmalar

BLOK 2 • İNCELEME • 15 DK

head ve tail: ilk ve son satırlar

```
print(kitaplar.head(3))
print(kitaplar.tail(2))
```

```
ÇIKTI
                         ad                yazar   yil    tur  puan  okundu
0              Tutunamayanlar            Oğuz Atay  1972  roman     5    True
1                  İnce Memed          Yaşar Kemal  1955  roman     4    True
2  Saatleri Ayarlama Enstitüsü  Ahmet Hamdi Tanpınar  1961  roman     5   False
                  ad                yazar   yil    tur  puan  okundu
6  Benim Adım Kırmızı          Orhan Pamuk  1998  roman     3    True
7            Semaver  Sait Faik Abasıyanık  1936   öykü     3   False
```

BLOK 2 • İNCELEME • 15 DK

info(): tablonun kimlik kartı

```
kitaplar.info()
```

```
ÇIKTI
<class 'pandas.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   ad      8 non-null      str
 1   yazar   8 non-null      str
 2   yil     8 non-null      int64
 3   tur     8 non-null      str
 4   puan    8 non-null      int64
 5   okundu  8 non-null      bool
dtypes: bool(1), int64(2), str(3)
memory usage: 764.0 bytes
```

BLOK 2 • İNCELEME • 15 DK

describe(): sayısal ve metin sütunların özeti

```
print(kitaplar.describe())
```

```
ÇIKTI
                yil      puan
count      8.000000  8.000000
mean    1962.125000  3.875000
std       24.485783  0.834523
min     1936.000000  3.000000
25%     1941.500000  3.000000
50%     1958.000000  4.000000
75%     1977.750000  4.250000
max     1998.000000  5.000000
```

```
print(kitaplar.describe(include="str"))
```

```
ÇIKTI
                       ad           yazar    tur
count                   8               8      8
unique                  8               7      2
top     Tutunamayanlar  Sabahattin Ali  roman
freq                    1               2      7
```

Pandas 2'de: include="object"

BLOK 2 • SEÇMEK • 20 DK

Sütun seçmek: df["ad"] ve df[["ad", "puan"]]

```
print(kitaplar["yazar"].head(3))
print(kitaplar[["ad", "puan"]].head(3))
print(kitaplar["puan"].mean())
```

```
ÇIKTI
0               Oğuz Atay
1            Yaşar Kemal
2    Ahmet Hamdi Tanpınar
Name: yazar, dtype: str
                            ad  puan
0               Tutunamayanlar     5
1                   İnce Memed     4
2  Saatleri Ayarlama Enstitüsü     5
3.875
```

Yazım

`df["puan"]`
`df[["ad", "puan"]]`
`df.puan`

Sonuç

tek sütun → Series
sütun listesi → DataFrame (çift köşeli parantez)
kısa yol; ad boşluksuz ve metot adıyla çakışmıyorsa

BLOK 2 • SEÇMEK • 20 DK

loc (adla) ve iloc (konumla)

```
kitaplar = kitaplar.set_index("ad")
print(kitaplar.loc["İnce Memed"])                          # bir satır (adla)
print(kitaplar.loc["Semaver", "yazar"])                    # hücre
print(kitaplar.iloc[0, 1], "|", kitaplar.iloc[-1].name)    # konumla
print(kitaplar.loc[["Semaver", "Tutunamayanlar"], ["yil", "puan"]])
```

```
ÇIKTI
yazar     Yaşar Kemal
yil              1955
tur             roman
puan                4
okundu           True
Name: İnce Memed, dtype: object
Sait Faik Abasıyanık
1972 | Semaver
                 yil  puan
ad
Semaver         1936     3
Tutunamayanlar  1972     5
```

Kural

loc[satır adı, sütun adı] • iloc[satır no, sütun no]. “l” label (ad), “i” integer (sayı).

BLOK 2 • DEĞIŞTIRMEK VE SIRALAMAK • 20 DK

Yeni sütun eklemek, sütun silmek, ad değiştirmek

```
kitaplar["yas"] = 2026 - kitaplar["yil"]            # yeni sütun: vektörel
kitaplar["klasik"] = kitaplar["yas"] ⩾ 50
kitaplar = kitaplar.drop(columns=["okundu"])
kitaplar = kitaplar.rename(columns={"yil": "ilk_yayin"})
print(kitaplar.head(4))
```

```
ÇIKTI
                          ad                yazar  ...  yas  klasik
0               Tutunamayanlar           Oğuz Atay  ...   54    True
1                   İnce Memed         Yaşar Kemal  ...   71    True
2  Saatleri Ayarlama Enstitüsü  Ahmet Hamdi Tanpınar  ...   65    True
3         Kürk Mantolu Madonna      Sabahattin Ali  ...   83    True

[4 rows x 7 columns]
```

BLOK 2 • DEĞIŞTIRMEK VE SIRALAMAK • 20 DK

Sıralamak: sort_values

```
print(kitaplar.sort_values("puan", ascending=False).head(3)[["ad", "puan"]])
cift = kitaplar.sort_values(["puan", "yil"], ascending=[False, True])
print(cift[["ad", "puan", "yil"]].head(3))
```

```
ÇIKTI
                          ad  puan
0               Tutunamayanlar     5
2  Saatleri Ayarlama Enstitüsü     5
1                   İnce Memed     4
                          ad  puan   yil
2  Saatleri Ayarlama Enstitüsü     5  1961
0               Tutunamayanlar     5  1972
3         Kürk Mantolu Madonna     4  1943
```

BLOK 2 • DEĞIŞTIRMEK VE SIRALAMAK • 20 DK

Saymak: value_counts, unique, nunique

```
print(kitaplar["yazar"].value_counts().head(3))
print(kitaplar["tur"].unique().tolist(), kitaplar["yazar"].nunique())
```

```
ÇIKTI
yazar
Sabahattin Ali    2
Oğuz Atay         1
Yaşar Kemal       1
Name: count, dtype: int64
['roman', 'öykü'] 7
```

value_counts = Ders 15'teki **GROUP BY yazar + COUNT(*)**'ın tek satırlık hali.

BLOK 2 • DEĞIŞTIRMEK VE SIRALAMAK • 20 DK

Filtreleme: NumPy’deki maske, şimdi sütun adlarıyla

```
okunmus = kitaplar[kitaplar["okundu"]]
print(okunmus[["ad", "puan"]])
print(kitaplar[(kitaplar["puan"] ⩾ 4) & (kitaplar["yil"] < 1960)]["ad"].tolist())
```

```
ÇIKTI
                    ad  puan
0        Tutunamayanlar     5
1            İnce Memed     4
3  Kürk Mantolu Madonna     4
4  Puslu Kıtalar Atlası     4
6    Benim Adım Kırmızı     3
['İnce Memed', 'Kürk Mantolu Madonna']
```

Kurallar Ders 19’daki gibi: koşullar **&**, **|**, **~** ile ve **parantez içinde**. Ayrıntılı filtreleme gelecek ders.

BLOK 2 • UYGULAMA VE ALIŞTIRMALAR • 20 DK

Uygulama: kitaplığımın raporu

```
kitaplar tablosu Bölüm 6'daki gibi
kitaplar["yas"] = 2026 - kitaplar["yil"]
print("Kitap sayısı:", len(kitaplar), "· Ortalama puan:", kitaplar["puan"].mean())
print("Okunma oranı: %", kitaplar["okundu"].mean() * 100)
print("En eski:", kitaplar.loc[kitaplar["yil"].idxmin(), "ad"])
print("Puanı 5 olanlar:", kitaplar.loc[kitaplar["puan"] == 5, "ad"].tolist())
print(kitaplar["tur"].value_counts().to_dict())
okunacak = kitaplar[~kitaplar["okundu"]].sort_values("puan", ascending=False)
print("Sıradaki okuma listesi:")
print(okunacak[["ad", "yazar", "puan"]].to_string(index=False))
kitaplar.to_csv("kitaplar.csv", index=False)
```

BLOK 2 • UYGULAMA VE ALIŞTIRMALAR • 20 DK

Raporun çıktısı

```
ÇIKTI
Kitap sayısı: 8 · Ortalama puan: 3.875
Okunma oranı: % 62.5
En eski: Semaver
Puanı 5 olanlar: ['Tutunamayanlar', 'Saatleri Ayarlama Enstitüsü']
{'roman': 7, 'öykü': 1}
Sıradaki okuma listesi:
                        ad               yazar  puan
Saatleri Ayarlama Enstitüsü Ahmet Hamdi Tanpınar     5
           Kuyucaklı Yusuf       Sabahattin Ali     3
                   Semaver Sait Faik Abasıyanık     3
```

BLOK 2 • SIK YAPILAN HATALAR

Seçme ve değiştirmede sık hatalar

Hatalı kod	Hata mesajı	Çözüm
`kitaplar["Puan"] # büyük P`	`KeyError: 'Puan'`	sütun adı birebir
`kitaplar["ad", "puan"]`	`KeyError: ('ad', 'puan')`	çift köşeli: [["ad", "puan"]]
`set_index sonrası loc[0, …]`	`KeyError: 0`	loc adla: iloc[0] ya da loc["…"]
`kitaplar["yeni"] = [1, 2, 3] # 8 satır`	`ValueError: Length of values (3) does not match length of index (8)`	uzunluk = satır sayısı

Hata vermeyen tuzak: **kitaplar.drop(columns=[…])** sonucunu kaydetmemek → tablo değişmez.

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 15 DK

Alıştırma 1, 2 ve 3

KOLAY • 4 dk

1 • Şehirler

İstanbul, Ankara, İzmir, Bursa, Antalya için (şehir, plaka, bölge) sütunlu bir DataFrame oluşturun; shape, dtypes ve Marmara'daki şehirler.

ORTA • 5 dk

2 • Not tablosu

5 öğrencilik (ad, vize, final) tablosu: ortalama sütunu (%40/%60), durum sütunu (≥ 50 Geçti), ortalamaya göre sıralı liste, sınıf ortalaması.

ZOR • 6 dk

3 • Satış tablosu

Ders 19 satış matrisini mağaza ve ay adlarıyla DataFrame yapın; "Toplam" sütunu, en çok satan mağaza (idxmax), Mart'ta 120'yi geçenler.

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 1: şehirler

```
import pandas as pd

sehirler = pd.DataFrame({
    "sehir": ["İstanbul", "Ankara", "İzmir", "Bursa", "Antalya"],
    "plaka": ["34", "06", "35", "16", "07"],
    "bolge": ["Marmara", "İç Anadolu", "Ege", "Marmara", "Akdeniz"],
})
print(sehirler.shape)
print(sehirler.dtypes)
print(sehirler[sehirler["bolge"] == "Marmara"]["sehir"].tolist())
```

```
ÇIKTI
(5, 3)
sehir    str
plaka    str
bolge    str
dtype: object
['İstanbul', 'Bursa']
```

Çözüm 2: not tablosu

```
import pandas as pd

notlar = pd.DataFrame({"ad": ["Ece", "Mert", "Zeynep", "Can", "Deniz"],
                       "vize": [60, 75, 40, 90, 55], "final": [70, 85, 55, 95, 42]})
notlar["ortalama"] = notlar["vize"] * 0.4 + notlar["final"] * 0.6
notlar["durum"] = "Kaldı"
notlar.loc[notlar["ortalama"] ⩾ 50, "durum"] = "Geçti"
print(notlar.sort_values("ortalama", ascending=False).to_string(index=False))
print("Sınıf ortalaması:", notlar["ortalama"].mean().round(1))
```

```
ÇIKTI
    ad  vize  final  ortalama durum
   Can    90     95      93.0 Geçti
  Mert    75     85      81.0 Geçti
   Ece    60     70      66.0 Geçti
Zeynep    40     55      49.0 Kaldı
 Deniz    55     42      47.2 Kaldı
Sınıf ortalaması: 67.2
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 3: satış tablosu

```
import numpy as np
import pandas as pd

satis = np.array([[120, 135, 128, 150],
                  [ 98, 110, 105, 120],
                  [140, 138, 152, 160]])
df = pd.DataFrame(satis, index=["Kadıköy", "Bornova", "Çankaya"],
                  columns=["Oca", "Şub", "Mar", "Nis"])
df["Toplam"] = df.sum(axis=1)
print(df)
print("En çok satan:", df["Toplam"].idxmax())
print("Mart > 120:", df[df["Mar"] > 120].index.tolist())
```

```
ÇIKTI
Oca  Şub  Mar  Nis  Toplam
Kadıköy  120  135  128  150
  533
Bornova   98  110  105  120
  433
Çankaya  140  138  152  160
  590
En çok satan: Çankaya
Mart > 120: ['Kadıköy', 'Çankaya']
```

KAPANIŞ • 5 DK

Bugün ne öğrendik?

- **Series**: adlı tek sütun; **DataFrame**: adlı sütunlardan oluşan tablo; işlemler ada göre hizalanır.
- Oluşturma: sütun sözlüğü, sözlük listesi, NumPy + index/columns.
- Tanıma: **head, tail, shape, info, describe, dtypes**.
- Seçme: **df["s"]**, **df[["s1", "s2"]]**, **loc** (adla), **iloc** (konumla), **df[koşul]**.
- Değiştirme: **df["yeni"] = …**, **drop, rename, set_index, sort_values, value_counts** — sonucu kaydetmeyi unutmayın.

KAPANIŞ • ÖDEV • TESLIM: BIR SONRAKI DERS

Ödev

1 • ZORUNLU

Film tablosu

Ders 14–15'teki filmlerinizi (en az 8) DataFrame yapın: yönetmen başına film sayısı, en yüksek puanlı 3 film, "yas" sütunu, filmler.csv olarak kaydedin.

2 • ZORUNLU

Harcama tablosu

Bir haftalık harcamalarınızı (tarih, kategori, tutar) en az 10 satırla girin: toplam, kategori sayıları (value_counts), en büyük harcama satırı (idxmax + loc).

3 • BONUS

Excel'e hazırlık

python -m pip install openpyxl kurun; harcama tablonuzu df.to_excel("harcamalar.xlsx", index=False) ile kaydedip Excel'de açın.

Teslim: ders20_odev.py ya da .ipynb. Teslim yöntemi: [teslim yöntemi]

KAPANIŞ • GELECEK DERS: HAFTA 11 • GÜN 1

Pandas 2: dosyadan okuma ve filtreleme

Bugün tabloyu elle yazdık ve kitaplar.csv olarak kaydettik. Gelecek ders gerçek dosyalardan okuyoruz:

- read_csv, read_excel ve SQLite'tan read_sql
- Ayırıcı, kodlama (encoding), başlık ayarları
- Filtreleme: isin, between, str.contains, query
- Açık veri portallarından gerçek bir veri seti

Önizleme

```
import pandas as pd

kitaplar = pd.read_csv("kitaplar.csv")
print(kitaplar.shape)
print(kitaplar.loc[0, "ad"])
```

```
(8, 7)
Tutunamayanlar
```

KAYNAKLAR

Bu dersin konuları resmi belgelerde

Ne için?	Adres
10 minutes to pandas	pandas.pydata.org/docs/user_guide/10min.html
Veri yapıları: Series ve DataFrame	pandas.pydata.org/docs/user_guide/dsintro.html
Seçme: loc ve iloc	pandas.pydata.org/docs/user_guide/indexing.html
Başlangıç eğitimleri	pandas.pydata.org/docs/getting_started/intro_tutorials